

Terraform Web Server Automation

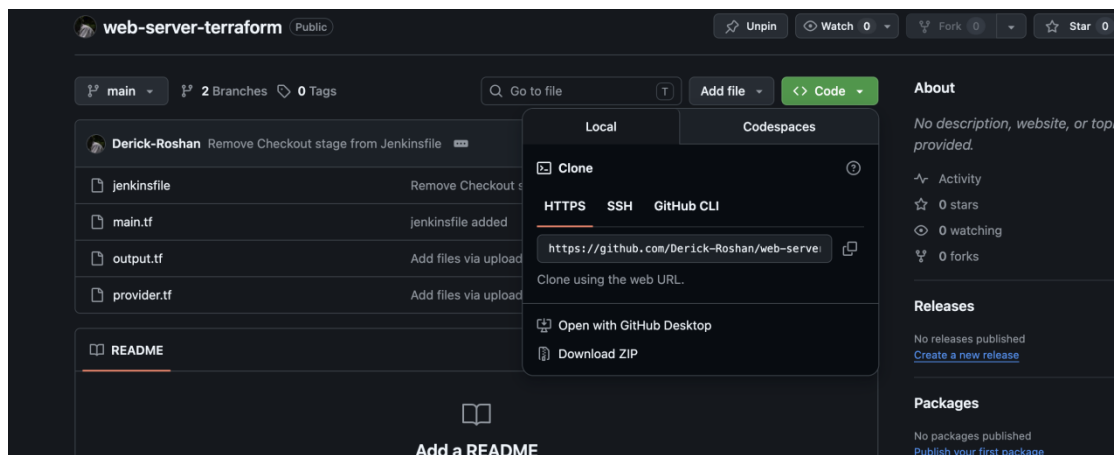
Automated EC2 provisioning using Terraform.

Technologies: Terraform, AWS, Git

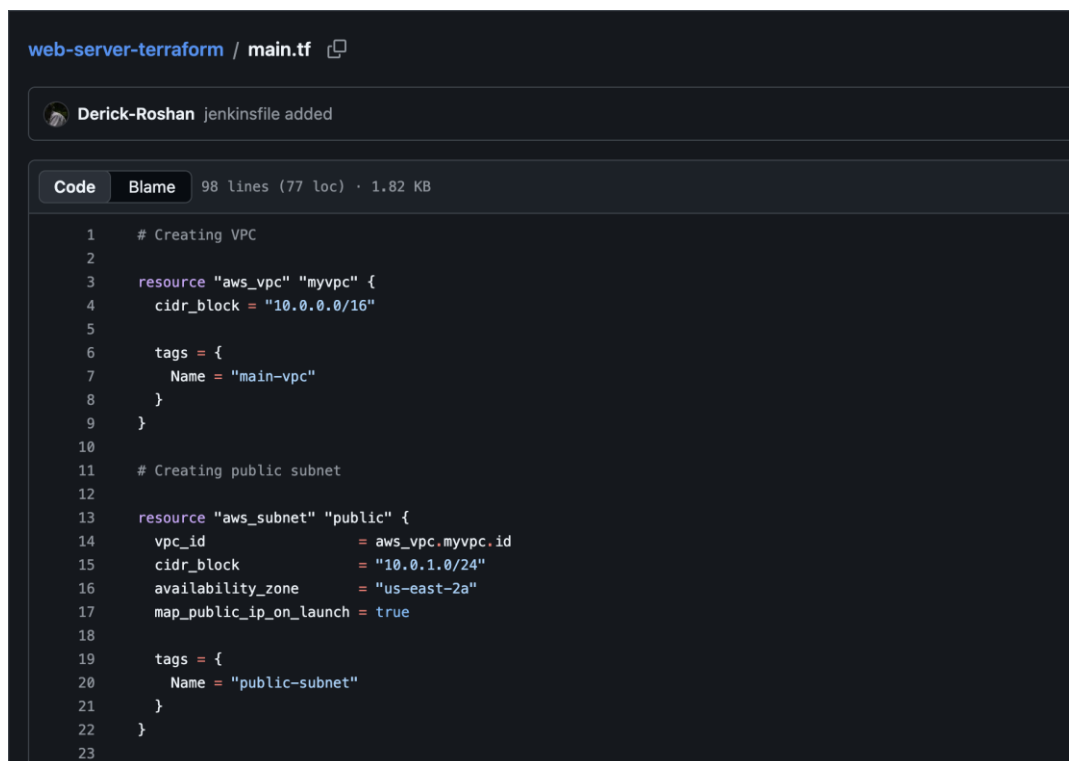
Repository: <https://github.com/Derick-Roshan/web-server-terraform>

Screenshots:

This is my terraform files stored in Github repo.



Main.tf



web-server-terraform / main.tf

Code Blame 98 lines (77 loc) · 1.82 KB

```
23
24 # Internet gateway
25
26 resource "aws_internet_gateway" "web-igw" {
27   vpc_id = aws_vpc.myvpc.id
28 }
29
30 # route table
31
32 resource "aws_route_table" "public_rt" {
33   vpc_id = aws_vpc.myvpc.id
34
35   route {
36     cidr_block = "0.0.0.0/0"
37     gateway_id = aws_internet_gateway.web-igw.id
38   }
39 }
40
41 resource "aws_route_table_association" "public_assoc" {
42   subnet_id      = aws_subnet.public.id
43   route_table_id = aws_route_table.public_rt.id
44 }
45
46 # security groups
47
48 resource "aws_security_group" "web_sg" {
49   name     = "web-sg"
50   vpc_id   = aws_vpc.myvpc.id
51
52   ingress {
53     from_port = 22
54     to_port   = 22
```

```
44 }
45
46 # security groups
47
48 resource "aws_security_group" "web_sg" {
49   name     = "web-sg"
50   vpc_id   = aws_vpc.myvpc.id
51
52   ingress {
53     from_port = 22
54     to_port   = 22
55     protocol  = "tcp"
56     cidr_blocks = ["0.0.0.0/0"]
57   }
58
59   ingress {
60     from_port = 80
61     to_port   = 80
62     protocol  = "tcp"
63     cidr_blocks = ["0.0.0.0/0"]
64   }
65
66   egress {
67     from_port = 0
68     to_port   = 0
69     protocol  = "-1"
70     cidr_blocks = ["0.0.0.0/0"]
71   }
72 }
73
74 # EC2 instance
```

```

73
74 # EC2 instance
75
76 resource "aws_instance" "web-server" {
77     ami             = "ami-09256c524fab91d36"
78     instance_type   = "t2.micro"
79     subnet_id       = aws_subnet.public.id
80     vpc_security_group_ids = [aws_security_group.web_sg.id]
81     key_name         = "spider"
82
83     user_data = <<-EOF
84     #!/bin/bash
85     yum install httpd -y
86     yum install git -y
87     yum install vim -y
88     git clone https://github.com/Derick-Roshan/Portfolio-website.git
89     cp -rf Portfolio-website/* /var/www/html/
90     systemctl enable httpd
91     systemctl start httpd
92     EOF
93
94
95     tags = {
96         Name = "web-server"
97     }
98 }

```

Provider.tf

web-server-terraform / provider.tf

Derick-Roshan Add files via upload

Code Blame 3 lines (3 loc) · 41 Bytes

```

1 provider "aws" {
2     region = "us-east-2"
3 }

```

Output.tf

web-server-terraform / output.tf

Derick-Roshan Add files via upload

Code Blame 4 lines (4 loc) · 132 Bytes

```

1 output "ec2_public_ip" {
2     description = "Public IP address of the EC2 instance"
3     value       = aws_instance.web-server.public_ip
4 }

```

Now I am opening the terminal and cloning the project to my local.

```
Last login: Sun Jun 28 10:53:43 on console
/Users/r/.zprofile:1: no such file or directory: /opt/homebrew/bin/brew
r@Roshans-MacBook-Pro ~ % ls
app.log          Desktop          Empty            Music            python-projects
Applications     Documents        Library          Pictures         streamlit.log
BrawlhallaReplays Downloads        Movies          Public           VirtualBox VMs
r@Roshans-MacBook-Pro ~ % cd Documents
r@Roshans-MacBook-Pro Documents % ls
Adobe            jenkinsfile-repo      sample-linux
Application_server_terraform Linux                Second-terraform-project
basic_python.py  Portfolio-website     terraform-modules
Demo-terraform   Predictive_maintanance Zoom
Details          Resume
r@Roshans-MacBook-Pro Documents % git clone https://github.com/Derick-Roshan/web-server-terraform.git
Cloning into 'web-server-terraform'...
remote: Enumerating objects: 76, done.
remote: Counting objects: 100% (76/76), done.
remote: Compressing objects: 100% (73/73), done.
remote: Total 76 (delta 23), reused 7 (delta 2), pack-reused 0 (from 0)
Receiving objects: 100% (76/76), 26.33 KiB | 1.88 MiB/s, done.
Resolving deltas: 100% (23/23), done.
r@Roshans-MacBook-Pro Documents % ls
Adobe            jenkinsfile-repo      sample-linux
Application_server_terraform Linux                Second-terraform-project
basic_python.py  Portfolio-website     terraform-modules
Demo-terraform   Predictive_maintanance web-server-terraform
Details          Resume                Zoom
r@Roshans-MacBook-Pro Documents %
```

Now initiating the terraform. (terraform init)

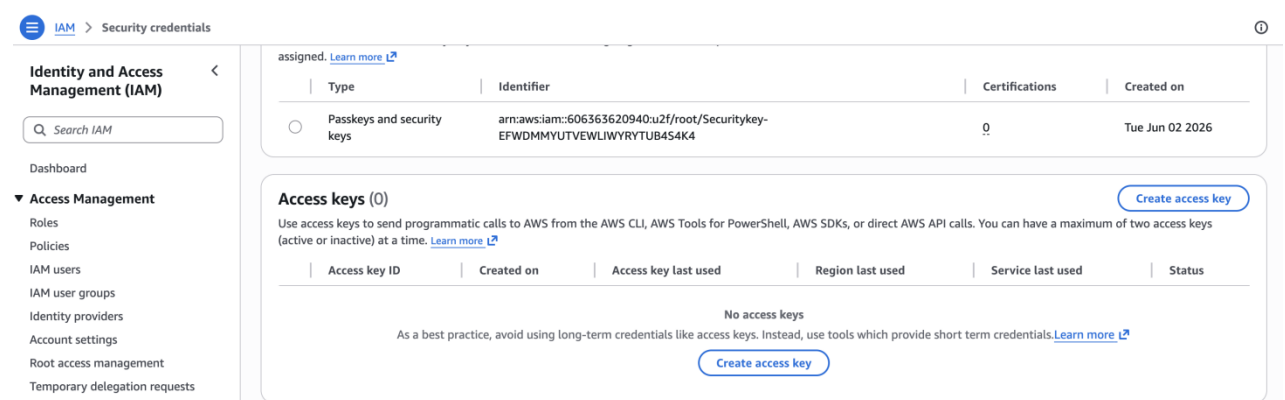
```
r@Roshans-MacBook-Pro Documents % cd web-server-terraform
r@Roshans-MacBook-Pro web-server-terraform % ls
jenkinsfile  main.tf          output.tf        provider.tf
r@Roshans-MacBook-Pro web-server-terraform % terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.52.0...
- Installed hashicorp/aws v6.52.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
r@Roshans-MacBook-Pro web-server-terraform %
```

Now going to AWS to create the “access key” on “Security credentials”



The screenshot shows the AWS IAM console interface. On the left, there is a sidebar with navigation links: Identity and Access Management (IAM), Roles, Policies, IAM users, IAM user groups, Identity providers, Account settings, Root access management, and Temporary delegation requests. The main content area is titled 'Security credentials' and contains a table of access keys. The table has columns for Type, Identifier, Certifications, and Created on. One access key is listed with the identifier 'arn:aws:iam::606363620940:u2f/root/Securitykey-EFWDMMYUTVEWLIWYRYTUB454K4'. Below the table, there is a section for 'Access keys (0)' with a 'Create access key' button. The page also includes a top navigation bar with the AWS logo and a search bar.

IAM > Security credentials > Create access key

Step 1

Alternatives to root user access keys

Retrieve access key

Alternatives to root user access keys info

Root user access keys are not recommended
Root user access keys have long-term unlimited permissions that can't be restricted. Instead, when accessing AWS CLI, SDKs, or tools for local development with AWS, use the `aws login` command and your existing console credentials for access.
[Learn more about alternatives to root user access keys](#)

Continue to create access key?

☒ I understand creating a root access key is not a best practice, but I still want to create one.

Cancel
Create access key

IAM > Security credentials > Create access key

This is the only time that the secret access key can be viewed or downloaded. You cannot recover it later. However, you can create a new access key any time.

Step 1

Alternatives to root user access keys

Retrieve access key

Retrieve access key info

Access key
If you lose or forget your secret access key, you cannot retrieve it. Instead, create a new access key and make the old key inactive.

Access key
Secret access key

AKIAI2LQWSZGL3LSPEPF
***** Show

Access key best practices

- Never store your access key in plain text, in a code repository, or in code.
- Disable or delete access key when no longer needed.
- Enable least-privilege permissions.
- Rotate access keys regularly.

For more details about managing access keys, see the [best practices for managing AWS access keys](#).

Download .csv file
Done

Copy and paste the credentials into the terminal to configure the AWS CLI.

```
r@Roshans-MacBook-Pro web-server-terraform % terraform validate
Success! The configuration is valid.

r@Roshans-MacBook-Pro web-server-terraform % aws configure
AWS Access Key ID [*****K6X2]: AKIAI2LQWSZGL3LSPEPF
AWS Secret Access Key [*****kUI0]: 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
Default region name [us-east-2]:
Default output format [None]:
r@Roshans-MacBook-Pro web-server-terraform %
```

Terraform plan

```
+ dhcp_options_id = (known after apply)
+ enable_dns_hostnames = (known after apply)
+ enable_dns_support = true
+ enable_network_address_usage_metrics = (known after apply)
+ id = (known after apply)
+ instance_tenancy = "default"
+ ipv6_association_id = (known after apply)
+ ipv6_cidr_block = (known after apply)
+ ipv6_cidr_block_network_border_group = (known after apply)
+ main_route_table_id = (known after apply)
+ owner_id = (known after apply)
+ region = "us-east-2"
+ tags = {
  + "Name" = "main-vpc"
}
+ tags_all = {
  + "Name" = "main-vpc"
}
}
```

Plan: 7 to add, 0 to change, 0 to destroy.

Changes to Outputs:

```
+ ec2_public_ip = (known after apply)
```

Note: You didn't use the `-out` option to save this plan, so Terraform can't guarantee to take exactly these actions if you run `"terraform apply"` now.

```
r@Roshans-MacBook-Pro web-server-terraform %
```

This is because I didn't give the permission for key-pair.

```
aws_instance.web-server: Creating...
aws_route_table_association.public_assoc: Creation complete after 1s [id=rtbassoc-014a6a4f4a5498f91]

Error: creating EC2 Instance: operation error EC2: RunInstances, https response error StatusCode: 400, RequestID: 8d892-97aa-4b8d-90ff-c8c92d86b455, api error InvalidKeyPair.NotFound: The key pair 'spider' does not exist

with aws_instance.web-server,
on main.tf line 76, in resource "aws_instance" "web-server":
76: resource "aws_instance" "web-server" {

r@Roshans-MacBook-Pro web-server-terraform % cd
r@Roshans-MacBook-Pro ~ % ls
app.log      Desktop      Empty        Music        python-projects
Applications Documents    Library      Pictures     streamlit.log
BrawlhallaReplays Downloads    Movies       Public       VirtualBox VMS
r@Roshans-MacBook-Pro ~ % cd Downloads
r@Roshans-MacBook-Pro Downloads % chmod 400 spider.pem
r@Roshans-MacBook-Pro Downloads % cd
r@Roshans-MacBook-Pro ~ % cd Documents
r@Roshans-MacBook-Pro Documents % cd web-server-terraform
r@Roshans-MacBook-Pro web-server-terraform % terraform apply
aws_vpc.myvpc: Refreshing state... [id=vpc-06c3f27ca9fe35c33]
aws_internet_gateway.web-igw: Refreshing state... [id=igw-01b991baf85ec9077]
aws_subnet.public: Refreshing state... [id=subnet-0fe37a9fe561ff391]
aws_security_group.web_sg: Refreshing state... [id=sg-01e4d097c9eb14c4d]
aws_route_table.public_rt: Refreshing state... [id=rtb-0cae011a0afa08890]
aws_route_table_association.public_assoc: Refreshing state... [id=rtbassoc-014a6a4f4a5498f91]
```

Terraform apply:

```
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

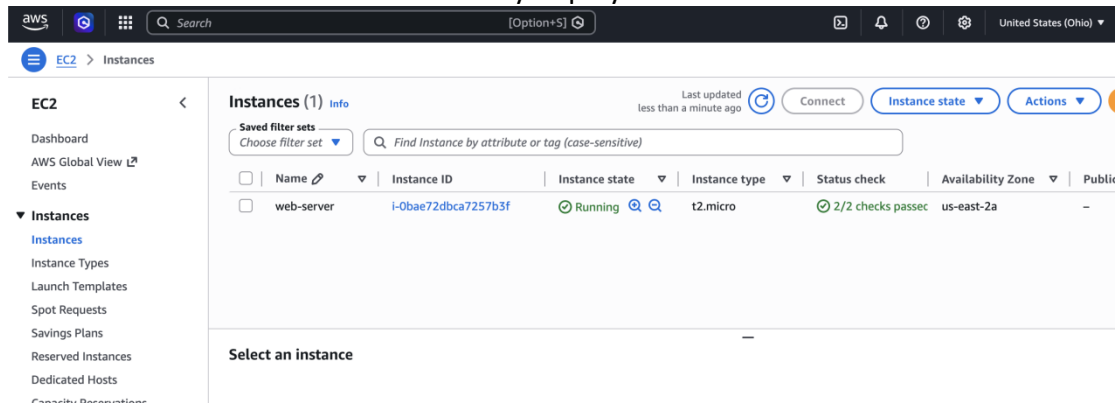
Enter a value: yes

aws_vpc.myvpc: Creating...
aws_vpc.myvpc: Creation complete after 6s [id=vpc-0b5faeee0dc7cd1da]
aws_internet_gateway.web-igw: Creating...
aws_subnet.public: Creating...
aws_security_group.web_sg: Creating...
aws_internet_gateway.web-igw: Creation complete after 3s [id=igw-019f8dad88ec9f33b]
aws_route_table.public_rt: Creating...
aws_route_table.public_rt: Creation complete after 3s [id=rtb-08845e20f5ba03d48]
aws_security_group.web_sg: Creation complete after 7s [id=sg-0b600534d5214319a]
aws_subnet.public: Still creating... [00m10s elapsed]
aws_subnet.public: Creation complete after 13s [id=subnet-0a0b41b8e1f6a3838]
aws_route_table_association.public_assoc: Creating...
aws_instance.web-server: Creating...
aws_route_table_association.public_assoc: Creation complete after 1s [id=rtbassoc-0fe336b82f7f05449]
aws_instance.web-server: Still creating... [00m10s elapsed]
aws_instance.web-server: Still creating... [00m20s elapsed]
aws_instance.web-server: Still creating... [00m30s elapsed]
aws_instance.web-server: Creation complete after 37s [id=i-0bae72dbca7257b3f]

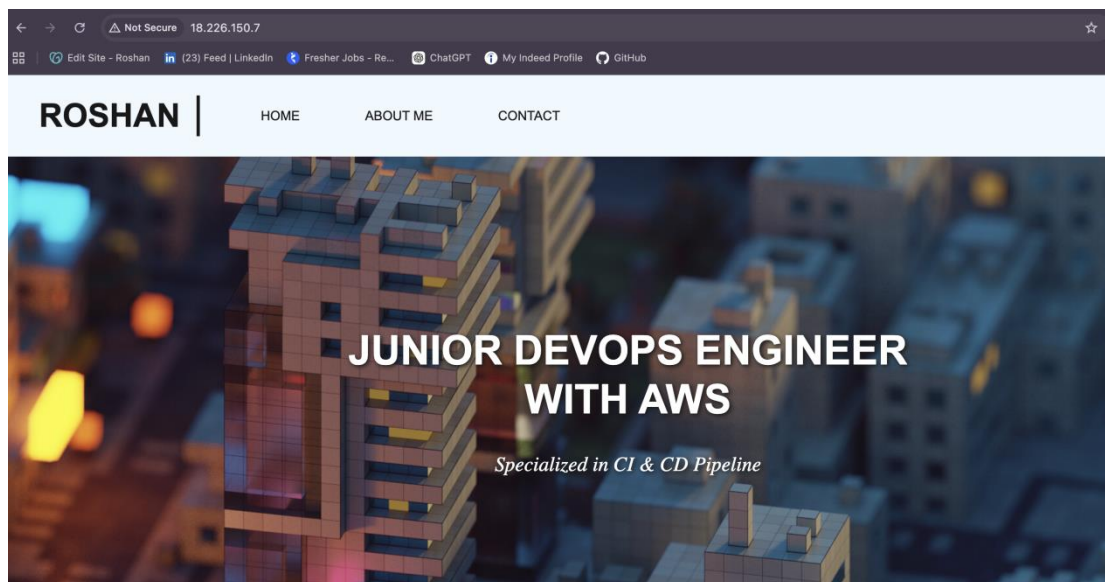
Apply complete! Resources: 7 added, 0 changed, 0 destroyed.

Outputs:
ec2_public_ip = "18.226.150.7"
r@Roshans-MacBook-Pro web-server-terraform %
```


All the resources have been successfully deployed.



Here you go!



Terraform destroy:

```
Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

aws_route_table_association.public_assoc: Destroying... [id=rtbassoc-0fe336b82f7f05449]
aws_instance.web-server: Destroying... [id=i-0bae72dbca7257b3f]
aws_route_table_association.public_assoc: Destruction complete after 2s
aws_route_table.public_rt: Destroying... [id=rtb-08845e20f5ba03d48]
aws_route_table.public_rt: Destruction complete after 2s
aws_internet_gateway.web-igw: Destroying... [id=igw-019f8dad88ec9f33b]
aws_instance.web-server: Still destroying... [id=i-0bae72dbca7257b3f, 00m10s elapsed]
aws_internet_gateway.web-igw: Still destroying... [id=igw-019f8dad88ec9f33b, 00m10s elapsed]
aws_instance.web-server: Still destroying... [id=i-0bae72dbca7257b3f, 00m20s elapsed]
aws_internet_gateway.web-igw: Still destroying... [id=igw-019f8dad88ec9f33b, 00m20s elapsed]
aws_internet_gateway.web-igw: Destruction complete after 24s
aws_instance.web-server: Still destroying... [id=i-0bae72dbca7257b3f, 00m30s elapsed]
aws_instance.web-server: Destruction complete after 36s
aws_subnet.public: Destroying... [id=subnet-0a0b41b8e1f6a3838]
aws_security_group.web_sg: Destroying... [id=sg-0b600534d5214319a]
aws_subnet.public: Destruction complete after 1s
aws_security_group.web_sg: Destruction complete after 2s
aws_vpc.myvpc: Destroying... [id=vpc-0b5faeee0dc7cd1da]
aws_vpc.myvpc: Destruction complete after 1s

Destroy complete! Resources: 7 destroyed.
r@Roshans-MacBook-Pro web-server-terraform %
```

